

AI Engineer Examination Project

Offline AI System for Arabic Patient Feedback Analysis



Prepared by: Abbass Zahreddine

Position: AI Engineer Candidate

Exam Title: AI Engineer Examination Project

Environment: Offline, Python-Based NLP and Speech AI System

This report summarizes the design, implementation, and evaluation of an offline AI system for analyzing Arabic patient feedback using NLP and speech recognition models.

Project Overview

This report provides a comprehensive documentation of the development and evaluation process carried out in this project. It summarizes the datasets used, the database design and API integration, and the experiments conducted on multiple models for different natural language processing tasks. The work completed is organized as follows:

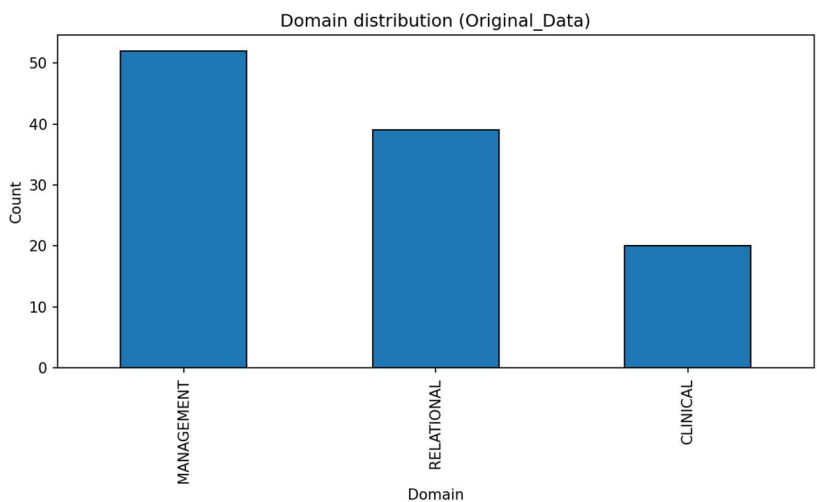
1. **Dataset Used** — Description of the datasets collected, cleaned, and prepared for model training.
2. **Database and API** — Explanation of the database structure and its integration through an API to support data storage and retrieval.
3. **Models Training and Testing**
 - a. **Named Entity Recognition (NER) Models** — Selection, training, testing, and comparison of different NER approaches.
 - b. **Classification Models** — Development, training, and evaluation of models for sentiment and aspect-based classification.
 - c. **Speech-to-Text (STT) Models** — Selection, fine-tuning, and performance evaluation of various speech recognition systems.

The Dataset

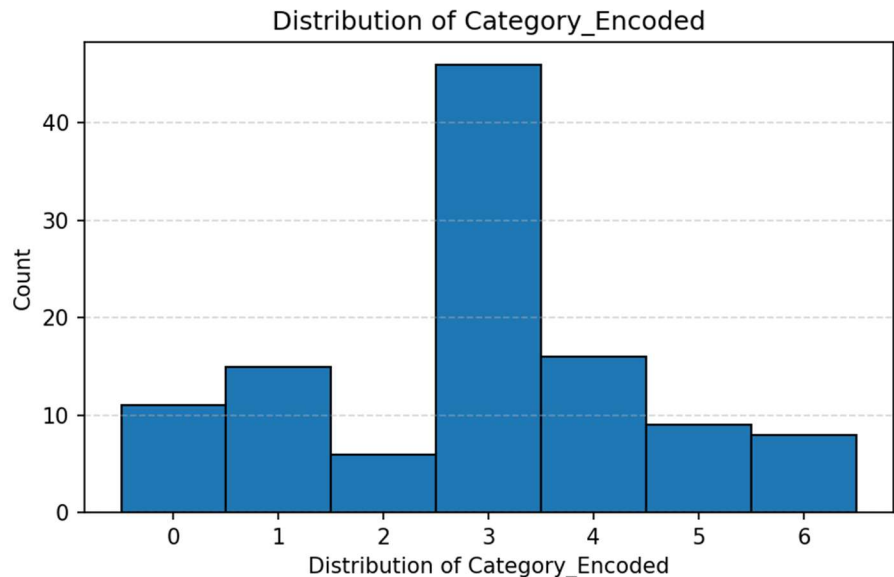
The dataset contains only 111 records, which makes it very small for reliable statistical analysis. This limited size highlights potential challenges such as data sparsity and class imbalance when building predictive models.

Domain Distribution

The domain distribution provides a clear view of where patient concerns are concentrated. It helps identify which areas of hospital operations may require more attention or improvement based on patient feedback



Category Distribution



The category distribution shows that the dataset is heavily imbalanced. For example, “Institutional Processes | Bureaucracy” and “Institutional Processes | Delay -Access” alone account for 38 of the 111 records, while many other categories, such as “Respect & Patient Rights | Rights” or “Safety | Error - Diagnosis,” appear only once. Several categories have fewer than 10 records, including “Quality of Care | Examination & Monitoring” (7 records) and “Environment | Accomodation” (6 records), which makes it extremely difficult to train a model to recognize these classes reliably. This imbalance can cause the model to be biased toward the dominant categories and struggle with underrepresented classes, reducing overall performance and generalization.

For model training, all categorical fields, including Domain and Category, were encoded into numeric values to allow the classification algorithms to process the data efficiently. This ensures consistency between text labels and model inputs.

Challenges for Model Training

The small dataset size (111 records) and severe class imbalance make model training difficult. Rare categories with fewer than 10 records, such as “Quality of Care | Examination & Monitoring,” may lead to overfitting and biased predictions toward dominant classes. Strategies like oversampling, class weighting, or data augmentation may be needed to improve performance and generalization.

Database and It’s API

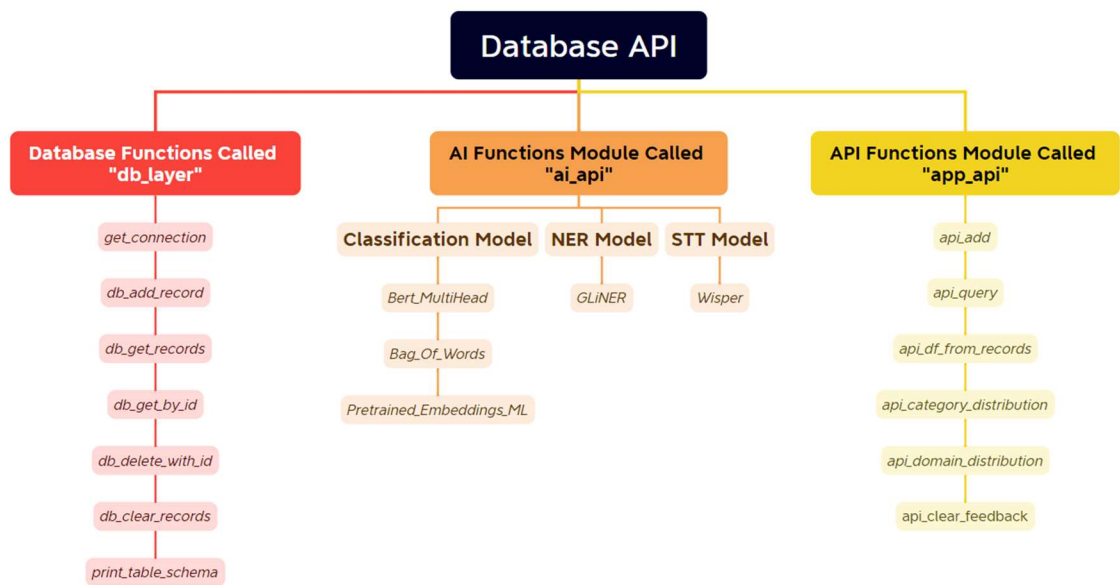


Figure 1: Modular structure of the project backend. The diagram shows the three main modules—db_layer, ai_api, and app_api—along with their internal functions and interactions. This visual illustrates the separation of responsibilities and the flow of operations

The project’s backend is organized into three main modules: **db_layer**, **ai_api**, and **app_api**. Each module encapsulates a specific layer of functionality. The **db_layer** handles all database interactions, including querying, inserting, and updating feedback records. The **ai_api** module manages the AI models, performing tasks such as classification, sentiment analysis, and NER. The **app_api** module integrates the previous two layers, exposing functions that are directly used by the **UI** to interact with both the database and the AI models seamlessly.

To illustrate this structure, an **XMind diagram** is provided, showing each module and its inner functions. This visual representation helps clarify the modular organization and the flow of function calls between layers.

The underlying **database schema** is implemented in **PostgreSQL**. An image exported from **pgAdmin4** shows the **hospital_feedback** table, including all columns and their types. The database stores textual patient feedback along with metadata such as hospital name, department, staff role, timestamp, and aspect-level sentiment labels. Together with the module diagram, this image provides a complete overview of how the application’s backend is structured and how data flows from storage to analysis.

Models Training and Testing

Classification Models

Multi-Head BERT-Based Classifier

Architecture:

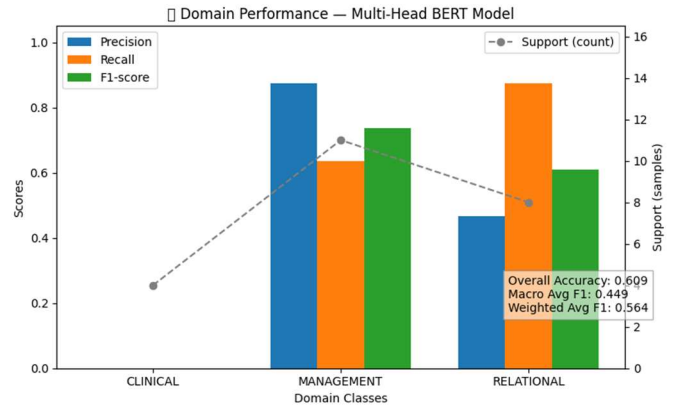
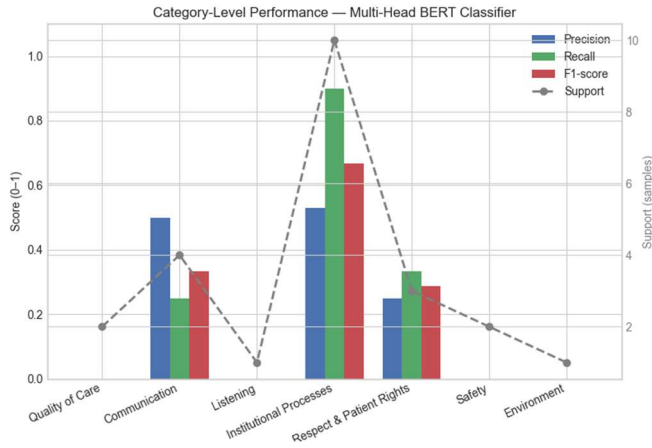
The project employs a multi-head BERT-based classifier that performs aspect-level sentiment analysis and classification simultaneously on patient feedback. Using `aubmindlab/bert-base-arabertv2` as the base model, it extracts the hidden representation of the [CLS] token from the input text and passes it through five separate linear heads, each corresponding to a hospital service aspect—Pricing, Appointments, Medical Staff, Customer Service, and Emergency Services. For each aspect, the model predicts a sentiment label coded as 0 for Positive, 1 for Negative, 2 for Neutral, and 3 for Not Mentioned, producing a vector that represents all aspects in one pass. This architecture allows the system to efficiently handle multiple aspects at once, enabling both classification and sentiment analysis within a single model, which is particularly suitable for healthcare feedback that often covers multiple service dimensions in the same text.

```
# -----  
# Multi-Head Model  
# -----  
class MultiHeadBERT(nn.Module): 1 usage  
    def __init__(self, model_name, domain_classes, category_classes):  
        super().__init__()  
        self.bert = AutoModel.from_pretrained(model_name)  
        hidden_size = self.bert.config.hidden_size  
        self.domain_head = nn.Linear(hidden_size, domain_classes)  
        self.category_head = nn.Linear(hidden_size, category_classes)  
  
    def forward(self, input_ids, attention_mask):  
        outputs = self.bert(input_ids=input_ids, attention_mask=attention_mask)  
        pooled_output = outputs.last_hidden_state[:, 0, :] # CLS token  
        domain_logits = self.domain_head(pooled_output)  
        category_logits = self.category_head(pooled_output)  
        return domain_logits, category_logits
```

AI System for Arabic Patient Feedback – Al-Rassoul Hospital

Category	P	R	F1	Support
Quality of Care	0.00	0.00	0.00	2
Communication	0.50	0.250	0.33	4
Listening	0.00	0.00	0.00	1
Institutional Processes	0.53	0.90	0.67	10
Respect & Patient Rights	0.25	0.33	0.29	3
Safety	0.00	0.00	0.00	2
Environment	0.00	0.00	0.00	1
Accuracy	0.48			
F1	0.18			
Avg F1	0.39			

Domain	P	R	F1	Support
Clinical	0.00	0.00	0.00	4
Management	0.88	0.64	0.74	11
Relational	0.47	0.88	0.61	8
Accuracy	0.609			
Macro Avg F1	0.449			
Weighted Avg F1	0.564			



Performance comparison of the Multi-Head BERT-Based Classifier across Category and Domain prediction tasks. Each group of bars represents Precision, Recall, and F1-score for a specific label, while the faint line indicates the relative support

The classifier's performance is uneven because of severe class imbalance. Frequent classes like MANAGEMENT and Institutional Processes dominate the learning, leading to high scores there, while rare categories like Listening, Quality of Care, or Environment get almost no correct predictions. The model is not failing completely — it captures dominant patterns — but it cannot generalize well to underrepresented classes with so few samples. Low F1-scores on these classes are expected under such data scarcity.

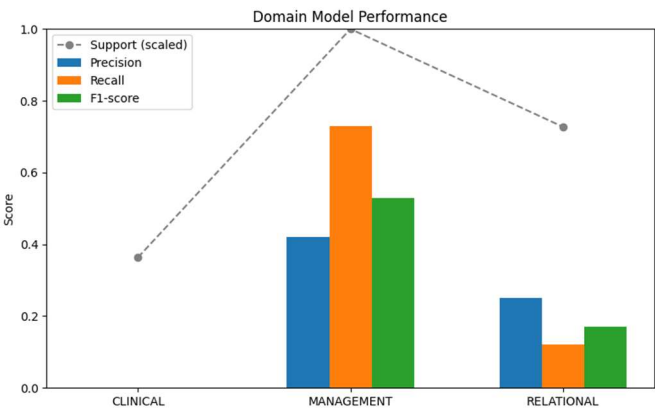
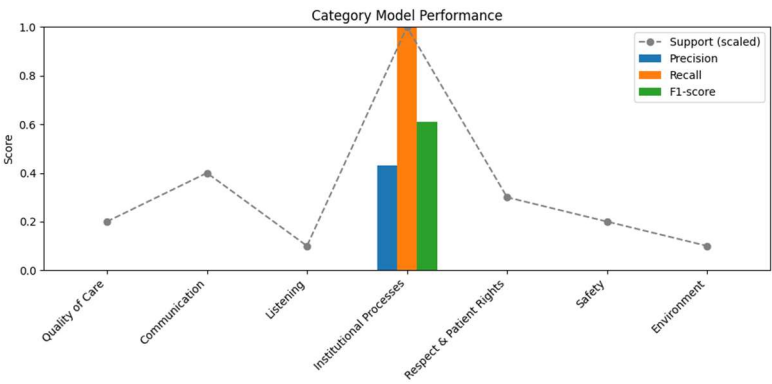
TF-IDF Classifier

Architecture

This model transforms Arabic patient feedback into TF-IDF features and uses a MultiOutputClassifier with Logistic Regression estimators, one per target aspect (Domain and Category). Each estimator predicts the sentiment label for its aspect independently. Optional class weights help mitigate class imbalance, enabling aspect-level predictions in a single, efficient model.

Category	P	R	F1	Support
Quality of Care	0.00	0.00	0.00	2
Communication	0.00	0.00	0.00	4
Listening	0.00	0.00	0.00	1
Institutional Processes	0.43	1.00	0.61	10
Respect & Patient Rights	0.00	0.00	0.00	3
Safety	0.00	0.00	0.00	2
Environment	0.00	0.00	0.00	1
Accuracy	0.435			
F1	0.09			
Avg F1	0.26			

Domain	P	R	F1	Support
Clinical	0.00	0.00	0.00	4
Management	0.42	0.73	0.53	11
Relational	0.25	0.12	0.17	8
Accuracy	0.391			
Macro Avg F1	0.23			
Weighted Avg F1	0.31			



Performance of the Domain and Category models showing precision, recall, and F1-score for each class. The grey dashed line indicates the scaled support (number of samples per class), highlighting severe class imbalance affecting model performance.

AI System for Arabic Patient Feedback – Al-Rassoul Hospital

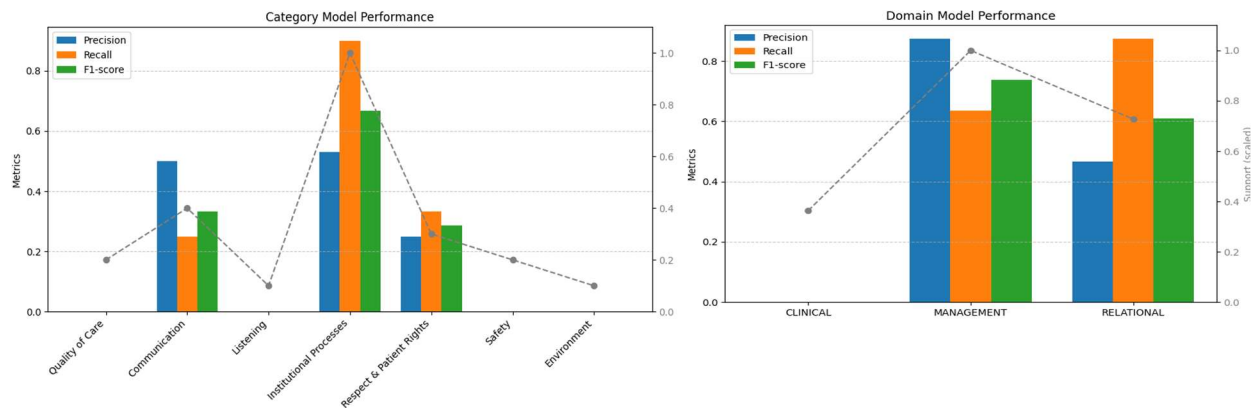
he classifier’s performance is **highly imbalanced**. Frequent classes like *MANAGEMENT* and *Institutional Processes* achieve moderate scores, but rare classes get nearly zero. Accuracy and F1-scores are low overall, reflecting **poor generalization** on underrepresented classes. This model captures dominant patterns but is unreliable for minority categories without data augmentation or class rebalancing.

Pretrained Embeddings ML

This model encodes Arabic patient feedback using pretrained Sentence-BERT (SBERT) embeddings. A separate Logistic Regression classifier is trained for each aspect (e.g., Pricing, Appointments, Medical Staff, Customer Service, Emergency Services). Predictions are made from the embeddings, and performance is evaluated using Accuracy, Precision, Recall, F1-score, and mAP, reporting overall mean F1 and mAP. The saved model and embedder allow efficient local inference. Compared to TF-IDF, SBERT embeddings capture semantic and contextual nuances in Arabic text, enabling more robust predictions for varied phrasing and sentiment.

Category	P	R	F1	Support
Quality of Care	0.000	0.000	0.000	2
Communication	0.500	0.250	0.333	4
Listening	0.000	0.000	0.000	1
Institutional Processes	0.529	0.900	0.667	10
Respect & Patient Rights	0.250	0.333	0.286	3
Safety	0.000	0.000	0.000	2
Environment	0.000	0.000	0.000	1
Accuracy	0.478			
F1	0.184			
Avg F1	0.385			

Domain	P	R	F1	Support
Clinical	0.000	0.000	0.000	4
Management	0.875	0.636	0.737	11
Relational	0.467	0.875	0.609	8
Accuracy	0.609			
Macro Avg F1	0.449			
Weighted Avg F1	0.564			

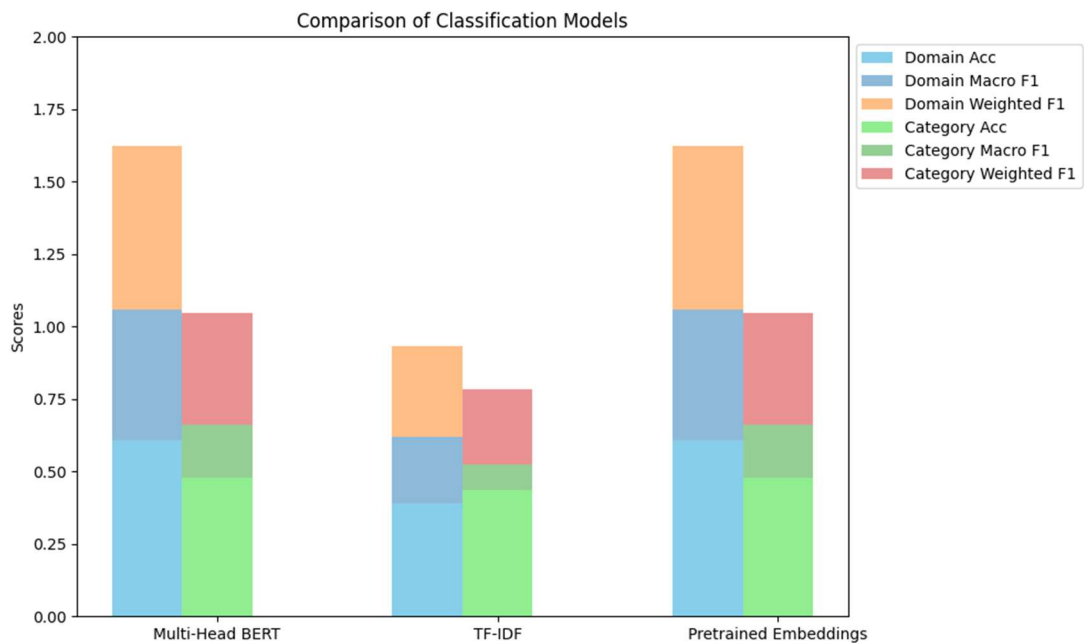


AI System for Arabic Patient Feedback – Al-Rassoul Hospital

Pretrained Embeddings ML model performance for Domain and Category prediction. Bars represent Precision, Recall, and F1-score per class, while the overlaid line shows class support (sample count), illustrating how class imbalance impacts performance

The model captures frequent classes like MANAGEMENT and Institutional Processes fairly well, but rare classes such as Listening, Quality of Care, Safety, and Environment perform poorly due to severe imbalance, showing that the classifier struggles to generalize to underrepresented classes

Model Comparison



After comparing the three models, **Pretrained Embeddings ML** is the best choice for integration into `ai_api`. It achieves similar performance to Multi-Head BERT while being lighter, semantically richer, and more robust to varied Arabic phrasing. TF-IDF + Logistic Regression is fast but underperforms and should only be considered if computational resources are severely limited.

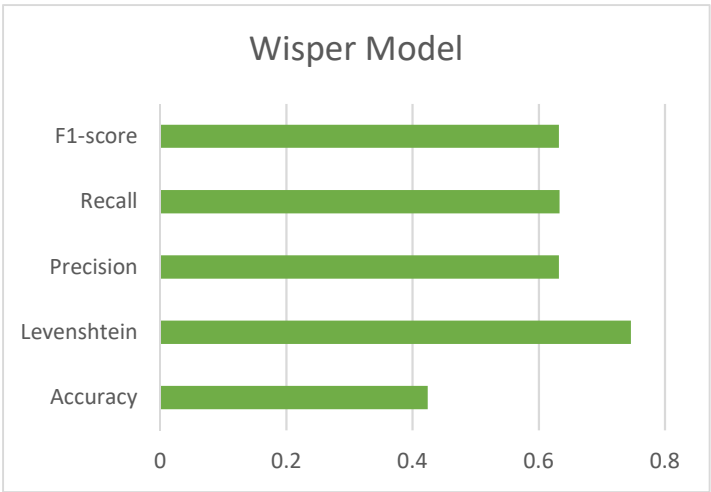
NER Model: GLiNER

The NER model performs very well for **patient name** and **date**, capturing most entities correctly. Performance drops for **doctor name**, likely due to fewer examples or higher variability in naming. Overall, the model is reliable for entity extraction but may require additional fine-tuning or data augmentation for doctor names.

ENTITY	ACCURACY
PATIENT NAME	0.892
DOCTOR NAME	0.586
DATE	0.946

STT Model: Wisper

I used the Wisper model. Also, I recorded 20 audio tracks in order to test the accuracy of this model. Here is the result.



Overall, Whisper performed **reliably for clear Arabic speech**, but accuracy dropped noticeably for clips containing **background noise, dialectal variation, or unclear pronunciation**. This makes it a suitable baseline STT model, with potential improvement through **fine-tuning or noise-robust preprocessing**.

Glimpse at the UI



AI-Rassoul Patient Feedback Analyzer

Collect and analyze patient feedback efficiently

Input Feedback

Enter feedback (or record voice):

Microphone Controls

Record high quality audio

00:00

Transcribe Audio

+ Add Feedback

NER Results

Date	Patient Name	Doctor Name

Run NER

Feedback Records

Filters

Category

Quality of Care x Communication x

Listening x Institutional Pro... x

Respect & Patien... x Safety x

Environment x

Domain

CLINICAL x MANAGEMENT x

RELATIONAL x

Refresh Download CSV Clear All

	Text	Category	Domain	Date
0	I felt ignored during my visit. Patient: Patient B Doctor: Dr. Sami Date: 2025-10-18	Institutional Processes	MANAGEMENT	2025-10-1
1	Doctor explained everything clearly. Patient: Patient B Doctor: Dr. Sami Date: 2025-10-18	Environment	CLINICAL	
2	I felt ignored during my visit. Patient: Patient A Doctor: Dr. Abbas Date: 2025-10-7	Environment	RELATIONAL	
3	Waiting time was too long. Patient: Patient D Doctor: Dr. Abbas Date: 2025-10-7	Listening	MANAGEMENT	
4	I appreciate the quick response. Patient: Patient A Doctor: Dr. Abbas Date: 2025-10-10	Communication	RELATIONAL	2025-10-1

Statistics